S'orienter

Personnes Programmes

	Documents	Données
Formats	docx, txt	xlsx, csv, R-DB

Et json, HTML, XML, jpg, pdf ?

XML et JSON

- Dans les 2 cas: un méta-langage = Définition de termes et d'une grammaire pour créer des documents/objets.
- XML vient du monde du document, du web, des échanges entre systèmes informatiques

Comparaison HTML - XML

HTML :

```
<SGML>
 --------------------
<html>
  <head><title>
   Dialogue extraterrestre
  </title></head>
  <body bgcolor="white"
        text="darkblue">
   <P> Salut la terre! </P>
   <P> Identifiez-vous
       et atterrissez! </P>
  </body>
</html>
```

XML :

```
<?xml --------  ?>
 --------------------
 <pièce_théâtre>
   <titre>
   Dialogue extraterrestre
   </titre>
   <conversation>
      <Salutation>
         Salut la terre!
      </Salutation>
      <Réponse>
        Identifiez-vous et
            atterrissez!
      </Réponse>
   </conversation>
 </pièce_théâtre >
```


Notions de base pour XML

- **Balise** : double étiquette qui introduit et clôt une information textuelle
  - <ville> et </ville>
- **Élément** : unité d'information délimitée par une balise
  - <ville> St Etienne </ville>
  - <ville> </ville> (élément vide)
- **Attribut** : couple (nom attribut, "valeur") qui qualifie un élément
  - <tel type="Fixe"> 04 77 22 43 13</tel>
  - <tel type='Mobile' pays="FR"> 06 72 43 22 31</tel>
  - <pays code='FR' /> **Élément vide avec attribut** (écriture raccourcie de <pays code='FR' > </>)
- **Commentaire** : <!-- C'est un commentaire -->

Notions de base pour XML (suite)

- Document bien formé
  - un élément racine qui est unique, et éventuellement compose d'autres éléments
  - des éléments correctement ouverts et fermés, sans imbrication

<a> <r> Jean Monnet </r> <v> St Etienne </v>

bien formé?

<a> <r> Jean Monnet <v> St Etienne </r> </v>

bien formé?

Notions de base pour XML

- Document bien formé
  - un élément racine qui est unique, et éventuellement compose d'autres éléments
  - des éléments correctement ouverts et fermés, sans imbrication

<a> <r> Jean Monnet </r> <v> St Etienne </v>

est bien formé

<a> <r> Jean Monnet <v> St Etienne </r> </v>

n'est pas bien formé car imbrication

Exemple de fichier XML

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<SALLE NO='2' PLACES='120'>
    <FILM>
        <TITRE>Vertigo</TITRE>
        <AUTEUR>Alfred Hitchcock</AUTEUR>
        <ANNEE>1958</ANNEE>
        <GENRE>Drame</GENRE>
        <PAYS>Etats Unis</PAYS>
        <RESUME>Scottie Ferguson, ancien inspecteur de police, est
        sujet au vertige depuis qu'il a vu mourir son collègue....
        </RESUME>
    </FILM>
    <SEANCES>
        <SEANCE>18:00</SEANCE>
        <SEANCE>22:00</SEANCE>
    </SEANCES>
</SALLE>
```


Créer des modèles de documents

- En JSON → JSON Schema
- En XML → DTD et XSD (XML Schema Description)

Un petit pas de côté : les espaces de nom

Espaces de nom

- Pour utiliser des balises
  - définies par d'autres
  - Accessibles ailleurs (à droite, à gauche, partout via le Web)

Espaces de nom (ns)

- Objectifs : pouvoir utiliser des termes provenant de plusieurs sources (modulariser), en évitant des conflits de nom
- Procédure : faire précéder les noms d'un identifiant spécifique avec un Unique Resource Identifier (URI)

- Exemple d'URI : http://www.chemin/maret
- Déclaration (avec xmlns)
  xmlns:MonNameSpace='http://www.chemin/maret'
- Utilisation en préfixe d'un nom d'élément XML
  < MonNameSpace:title>

Espaces de nom

- Exemple d'utilisation de namespace en XML

```
<book
xmlns:s='http://www.chemin/maret'
xmlns:html='http://www.w3.org/TR/REC-html40'>
   <s:title>XML</s:title>
   <s:author>O. Corby</s:author>
   <html:p> ...</html:p>
    <s:p>un autre contenu…</s:p>
</book>
```

Retour à :
Créer des modèles de documents

- XML → DTD et XDS (XML Schema Description)

Créer un document XML

Bases DTD
NS XSchema
Xlink Xpath
XSLT Xquery

Les éléments, contenus, valeurs d'attributs du doc XML doivent être exprimés en suivant un vocabulaire et des règles (grammaire)

Balises
<SALLE>, <FILM>, <SEANCES>, <SEANCE>

Attributs
<SALLE NO='2' PLACES='120'>

Enchainement de balises
<SEANCES>
<SEANCE>18:00</SEANCE>
<SEANCE>22:00</SEANCE>
</SEANCES>

Créer un document XML

Bases DTD
NS XSchema
Xlink Xpath
XSLT Xquery

Les éléments, contenus, valeurs d'attributs du doc XML doivent être exprimés en suivant un vocabulaire et des règles (grammaire)
Exemples
Balises
<SALLE>, <FILM>, <SEANCES>, <SEANCE>

Attributs
<SALLE NO='2' PLACES='120'>

Enchainement de balises
<SEANCES>
<SEANCE>18:00</SEANCE>
<SEANCE>22:00</SEANCE>
</SEANCES>

Le vocabulaire, la grammaire d'un document sont défini dans une DTD ou dans un XSD

XML : les DTD

DTD : document type definition

Le vocabulaire et la structure prédéfinie d'un document = une grammaire

DTD : document type definition

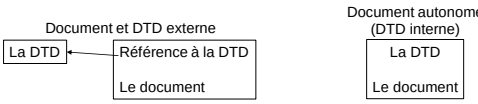
La DTD détermine

- Vocabulaire de balisage
- Structure des éléments imbriqués
- Méta-donnees définissant l'imbrication des balises
  - Obligatoire / Optionnel
  - Emboîtées / En séquence
  - Itérées

Note : Ancêtre de XML = SGML, contient aussi la notion de DTD.

DTD : document type definition

- Définitions
  - Un document est dit bien formé si c'est un fichier XML correct
  - Un document est valide s'il vérifie la DTD qui lui est attaché.
  - Un document est autonome (standalone) s'il contient sa propre DTD



Déclaration d'élément (1)

<!ELEMENT Nom TypeContenu>

- Nom : alphabétique (pas "xml" au début), sensible à la casse
- TypeContenu : éléments simples ou composés

cf. diapo suite

Éléments simples = Contenu simple = Types prédéfinis →3

EMPTY (élément vide = pas de données) <!ELEMENT pays EMPTY>
(#PCDATA) (parsed character data = des données qui seront analysées
<!ELEMENT nom (#PCDATA)> puis en xml <nom> Janin </nom>
ANY (tout type de données = des données qui non analysées)
<!ELEMENT doc ANY> puis en xml <doc> texte
…. </doc>

Déclaration d'élément (2)

<!ELEMENT Nom TypeContenu>

- TypeContenu : éléments simples (cf avant) ou composés.

Éléments composés = contenu composé d'éléments

- Opérateurs entre éléments (placés après l'élément)

+	(unaire)	1 ou plusieurs fois
*	(unaire)	0 ou plusieurs fois
?	(unaire)	0 ou 1 fois (optionnel)
|	(binaire)	ou
,	(binaire)	et (=suivi de, séquence, ordonné)
( )	regroupement d'éléments pour l'application d'opérateur	

<!ELEMENT personne ((nom, prenom+) | #PCDATA)>

Déclaration d'élément (2)

<!ELEMENT personne ((nom, prenom+) | #PCDATA)>

Ecrire des exemples de documents XML respectant cette ligne de DTD

Déclaration d'élément (2)

<!ELEMENT personne ((nom, prenom+) | #PCDATA)>

Ecrire des exemples de documents XML respectant cette ligne de DTD

```
<personne>
    <nom> Sartre </nom>
    <prenom> Jean </prenom>
    < prenom > Paul
</prenom>
    < prenom > Michel
</prenom>
</personne>
```

```
<personne>
    Alain Michel
</personne>
```

```
<personne>
</personne>
```


Déclaration d'élément (2 bis)

Exemple d'élément composés

<!ELEMENT MaBD (LesPersonnes, LesEntreprises)>
<!ELEMENT LesPersonnes (personne*)>

<!ELEMENT personne (prenom+, adresse?, ville?, tel*, email?)>
<!ELEMENT prenom, adresse, ville, tel, email (#PCDATA)>

```
<personne>
   <prenom> Paul </prenom>
   <prenom> Lucien </prenom>
   <adresse> … </adresse>
    <tel> … </tel> <tel> … </tel>
</personne>
```

Attribut d'un élément

- Les éléments peuvent posséder 0 ou n attributs
  - portent des informations (parfois non affichées, parfois réservées aux traitements**)**

<SALLE NO='2' PLACES='120'>

</SALLE>

Attributs d'un élément

Tout attribut est décrit par :
<!ATTLIST balise [nom-attr type-attr mode [valeur]]* >

- balise est le nom de la balise concernée
- nom-attribut est le nom de l'attribut
- type-attribut est un des types de données de base
- mode : #REQUIRED, #IMPLIED, #FIXED
  #REQUIRED : obligatoire
  #IMPLIED : facultatif (valeur par défaut)
  #FIXED : fixé à la valeur associée
- valeur est la valeur fixée et/ou une énumération : (v1 | v2 | ...)

*= répétition (on peut déclarer plusieurs attributs pour une même balise)

Types de base des attr.

- Types de données de base
  - CDATA : donnée entre " (non analysée par parseur)
  - NMTOKEN, NMTOKENS : lettre, chiffres et .-_:
  - (n1|n2|…): Énumération, séparateur |
  - ID, IDREF, IDREFS : attribut clé (un seul attribut de type ID dans un élément...) ou attribut dont la valeur fait référence a une clé
  - ENTITY, ENTITIES (cf. après)
  - NOTATION (cf. après)

Types de base des attr.

- Exemple :

Type enum sous-entendu

```
<!ELEMENT tel (#PCDATA)>
<!ATTLIST tel usage #REQUIRED ("personnel" | "professionnel" |
    "mobile") "personnel" >
<!ELEMENT entreprise (nom, ville)>
<!ATTLIST entreprise identifiant ID #REQUIRED>
<!ELEMENT employeur EMPTY>
<!ATTLIST employeur idEmp IDREF #REQUIRED>
```

Valeur par défaut

Valeur unique dans le doc XML. Pas un nombre

Exemple de DTD

```
<! la DTD de ma BD des personnes >
<!ELEMENT MaBD (LesPersonnes, LesEntreprises)>
<!ELEMENT LesEntreprises (entreprise+)>
<!ELEMENT LesPersonnes (personne+)>
<!ELEMENT personne (prenom, adresse ?, ville ?, tel*, mel ?)>
<!ELEMENT adresse (num, rue, ville)>
<!ELEMENT prenom (#PCDATA)>
<!ELEMENT ville (#PCDATA)>
<!ELEMENT tel (#PCDATA)>
<!ATTLIST tel usage #REQUIRED ("personnel" | "professionnel" |
    "mobile")>
<!ELEMENT mel (#PCDATA)>
<!ELEMENT num (#PCDATA)>
<!ELEMENT rue (#PCDATA)>
```

Note : une syntaxe héritée de SGML
une syntaxe non XML!

Rappel :
+ 1 ou plusieurs fois
? 0 ou 1 fois
***** 0,1 ou plusieurs fois
, séquence
| alternative

Produire le document XML

- Un document XML respecte une DTD (= document valide)
- La DTD est associée au document:
  - En interne au document (document « standalone »)
  - En externe au document (mot clé SYSTEM ou PUBLIC)

Exemple de fichier XML avec DTD interne :

```
<?xml version="1.0" encoding= "ISO-8859-1" ?>
<!DOCTYPE MesPersonnes [
<!- la DTD de ma BD des personnes ->                    DTD
<!ELEMENT MesPersonnes (personne+)>
<!ELEMENT personne (prenom, adresse ?, ville ?, tel*, mel ?)>
.....
<!ELEMENT rue (#PCDATA)>
]>
<![ Ma base de données des personnes [>
<MesPersonnes> ..... </MesPersonnes>                   Document
```

DTD et document XML

- Exemple de fichier XML avec DTD externe :

```
<xml version="1.0" encoding= "ISO-8859-1" ?>
<!DOCTYPE MesPersonnes SYSTEM "/home/DTD/MesPersonnes.dtd">
<!- Ma base de donnees des personnes ->
<MesPersonnes> ..... </MesPersonnes>
```

- ou encore :

```
<xml version="1.0" encoding= "ISO-8859-1" ?>
<!DOCTYPE MesPersonnes PUBLIC "http://unchemin/MesPersonnes.dtd">
<!- Ma base de donnees des personnes ->
<MesPersonnes> ..... </MesPersonnes>
```

Avec MesPersonnes.dtd qui contient la DTD :

```
<!- la DTD de ma BD des personnes ->
<!ELEMENT MesPersonnes (personne+)>
.....
```


Textes répétitifs : Entity General Entity

- Constante texte déclarée dans la DTD pour répétition dans document XML ou dans autre déclaration ENTITY
  <!ENTITY nom-entrep "Internet Solutions">

Appelé entre & et ; dans le XML

<Titre> Description de la société &nom-entrep; </Titre>
<!ENTITY nom-entrep-long " The &nom-entrep; Corp.">

Qqs General Entity pré-définies:
< (<), > (>), & (&), " ("), et ' (')

Textes répétitifs : Entity. Parameter Entity

- Fragment déclaré dans la DTD pour répétition dans la DTD (déclaration d'élément)

% et ; pour l'utiliser (respecter les espaces)

```
<!- Dans la DTD ->
<!ENTITY % adresse "num, rue, ville, codeP">
<!ELEMENT MaBD (LesPersonnes, LesEntreprises)>
  <!ELEMENT LesEntreprises (entreprise+)>
  <!ELEMENT LesPersonnes (personne+)>
  <!ELEMENT personne (nom, prenom, %adresse;, tel*, mel?,
    employeur)>
  <!ELEMENT rue (#PCDATA)>
```


Entity externes → permet la réutilisation

- Si le fragment de DTD se trouve dans fichier externe, il faut le déclarer:
  - Mot-clé PUBLIC pour accès distant

<!ENTITY % telephone PUBLIC "http://chemin/tel ">

  - Mot-clé SYSTEM pour un accès local

<!ENTITY % telephone SYSTEM "/home/..">

Permet d'utiliser %telephone; dans la DTD
<!ELEMENT %telephone; >

Limites des DTD

- Une DTD ne définit que la grammaire des balises.
  - Expression dans un autre langage que XML
  - Typage absent (ou presque) :
    - Types de base : texte uniquement
    - Pas de contrainte sur les caractères
    - Pas de contrainte de type sur les IDREFs
  - Modularité et re-utilisation trop pauvre
  - Pas de mécanisme de généralisation / spécialisation
  - Contraintes d'intégrité : identifiant trop pauvre, intégrité référentielle absente.

Mais où est l'acquis des bases de données (notion de schéma, clé étrangère, typage fort, etc.) ?